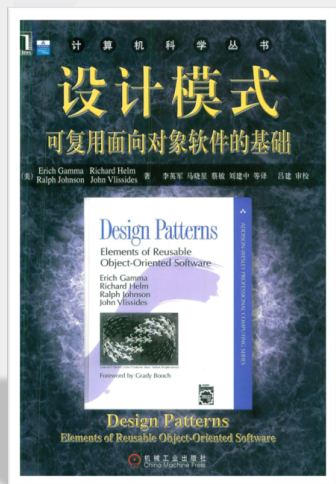




设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

单例模式（Embedded-Oriented Singleton）

1. 意图（Intent）

在系统中保证某类资源：

- 全局唯一实例
- 可控访问路径
- 受约束的生命周期
- 避免重复初始化

在嵌入式场景中，本质目标通常不是“对象唯一”，而是：

保证物理资源或内核核心控制结构的唯一性与一致性。

2. 动机（Motivation）

嵌入式系统具有天然的“单实例资源”特征：

- MCU 外设物理唯一（UART1、SPI2 等）
- 系统 Tick 唯一
- 调度器控制块唯一
- 当前任务指针唯一
- 内核状态机唯一

如果不进行结构约束：

- 可能重复初始化硬件

- 状态被多个模块无序修改
- 系统进入不可预测状态

典型问题示例

```
void uart_init(void);  
void uart_send(const char *data);
```

如果多个模块随意调用 `uart_init()`：

- 可能重复配置寄存器
- 打断正在进行的 DMA
- 破坏通信状态

嵌入式单例的核心目的：

把“资源唯一性”提升为架构约束。

3. 适用性 (Applicability)

在嵌入式系统中，以下场景适合单例：

3.1 架构级资源

- 调度器核心控制块
- 系统时间源
- 中断管理器
- 内核状态机

3.2 外设抽象层

- UART 驱动控制块
- SPI 总线管理器
- DMA 控制器
- Flash 驱动

3.3 协议栈核心

- TCP 活跃 PCB 链表
- 网络接口链表头
- BLE 协议栈全局状态

4. 结构 (Structure)

在 C 语言中没有 `class` 和 `private` 构造函数，因此结构通常表现为：

4.1 文件级静态实例

```
static struct uart_driver g_uart;
```

4.2 对外暴露访问接口

```
struct uart_driver* uart_get_instance(void)
{
    return &g_uart;
}
```

4.3 内部初始化控制

```
static int initialized = 0;

void uart_init(void)
{
    if (initialized)
        return;

    /* hardware configuration */
    initialized = 1;
}
```

5. 参与者 (Participants)

在嵌入式单例中，典型参与角色如下：

角色	描述
Singleton Instance	静态全局结构体
Access Interface	获取实例或访问 API
Client Modules	使用该资源的其他模块
Synchronization Mechanism (同步机制)	中断屏蔽 / 自旋锁 / 原子操作

6. 协作 (Collaborations)

6.1 单核 MCU

- 通过关中断保证一致性
- 无真正并行线程竞争

6.2 RTOS

- 调度器控制块在临界区更新
- 使用任务级同步机制

6.3 SMP 系统

- 使用 per-core 单例拆分竞争
- 或使用 spinlock 保护

7. 结果 (Consequences)

优点

1. 明确资源所有权
2. 防止重复初始化
3. 简化资源访问路径
4. 降低系统状态不一致风险
5. 减少动态内存使用

缺点

1. 强耦合
2. 难以单元测试
3. 难以做多实例仿真
4. 全局状态可能导致隐式依赖

嵌入式特有风险

- 中断与任务访问竞争
- SMP 下缓存一致性问题
- 初始化顺序依赖

8. 实现 (Implementation)

在嵌入式 C 中，常见实现方式有三类：

8.1 直接全局变量 (Linux 风格)

```
struct kernel_control_block kernel;
```

优点：简单直接

缺点：无封装

8.2 static + getter（模块化风格）

```
static struct scheduler sched;

struct scheduler* scheduler_instance(void)
{
    return &sched;
}
```

优点：

- 限制外部直接访问
- 可隐藏内部结构

8.3 每核心单例（SMP 优化）

```
DEFINE_PER_CPU(struct rq, runqueues);
```

特征：

- 每 CPU 唯一
- 降低锁竞争
- 提高可扩展性

8.4 并发控制策略

执行模型	控制方式
单核 + 中断	关中断
RTOS	临界区
SMP	spinlock
只读单例	不需要锁